— PROJECT REPORT —

SonicSentinel AI

AcousticX Intelligence

NextWave AI and ML



MENTOR
Hifza Toufiq

TEAM MEMBER
Haleema Ishaq
Student ID 1542292

TEAM MEMBER
Taha Mubashir Khan
Student ID 1544659

TEAM MEMBER
Huma kiran
Student ID 1542325

TEAM MEMBER
Atiq Imran
Student ID 1549120

Table of Contents

1. Project Overview
2. Requirements
3. System Design
4. Database Design
5. Diagrams
6. Audio and AI Design
7. Dataset and Training
8. Model Evaluation
9. Testing, Security and Privacy
10. Limitations and Future Enhancements

1. Project Overview

1.1 Problem Definition

Manual sound monitoring is slow, inconsistent, and difficult in noisy environments. Important events such as gunshots, glass breaking, panic screams, alarms, aggression, calls for help, and machinery faults can be missed or identified too late. SonicSentinel AI addresses this problem by automatically analyzing uploaded audio and live microphone input and turning sound events into classifications, confidence scores, severity levels, alerts, and reviewable records.

1.2 Background and Business Necessity

Sound-event monitoring is useful in manufacturing plants, public spaces, transport facilities, commercial buildings, homes, farms, service centers, and security-sensitive environments. The business need is to reduce continuous manual listening, improve response speed, create consistent event records, and support operators with evidence such as waveform, spectrogram, model confidence, event history, and alert status.

1.3 Proposed Solution

SonicSentinel AI is a Python-based web application with two audio inputs: uploaded files and live microphone windows. Audio is validated, preprocessed, segmented, and independently classified by a Python model and a Google Teachable Machine (GTM) audio model. Their predictions and confidence scores are compared. Audio quality and critical-event rules are then applied before the final result, alert, or manual-review decision is produced.

1.4 Purpose

The purpose is to build a reliable competition prototype for automatic sound-event detection, model comparison, real-time monitoring, critical-event recognition, and reporting. It is not intended to be a certified emergency-response or law-enforcement system.

1.5 Scope

- User registration, authentication, profiles, and role-based access.
- Uploaded audio analysis and live microphone monitoring.
- Validation, preprocessing, segmentation, waveform and spectrogram generation.
- Python ML classification and independent GTM classification.
- Confidence comparison, audio-quality analysis, repeated-event confirmation, severity and alerts.
- Manual review, event history, dashboards, analytics, reports, export, audit trail and model-version tracking.

1.6 Assumptions

- Users provide microphone permission before live monitoring.
- Training data is ethically sourced, labelled correctly, and balanced across the required classes.
- Python and GTM models use the same underlying dataset split but are trained independently.
- The deployment environment has sufficient CPU/RAM and browser microphone support.
- Thresholds and critical-event rules can be tuned by authorized administrators.

1.7 Constraints

Accuracy is affected by noise, microphone type, recording distance, echo, overlapping sounds, class similarity, dataset quality, and computational resources. Python and GTM may disagree because they are independently trained. Privacy, consent, bias across speakers/devices/environments, and storage limits must also be managed.

2. Requirements

2.1 Functional Requirements

Area	Requirement Summary
Authentication & RBAC	Register/login users and control access for normal users, reviewers, security operators, maintenance operators and administrators.
Audio Input	Upload WAV/MP3/FLAC/OGG/M4A files, batch upload where authorized, and start/stop live microphone monitoring.
Validation & Metadata	Validate format, size, duration, sampling rate, channels, integrity and usable signal; extract audio metadata.
Preprocessing	Resample, convert to mono, normalize amplitude, trim silence, reduce noise, segment, pad or truncate.
Dataset & Features	Maintain labelled dataset metadata, balance checks and augmentation; extract MFCC, Mel spectrogram, chroma, ZCR, RMS and spectral features.
Classification	Run the selected Python model and the GTM model independently and return confidence scores for all classes.
Comparison	Compare top predictions, confidence difference, top-two margin and model-consistency status.
Quality & Uncertainty	Classify audio quality and detect low confidence, overlapping sounds, disagreement and uncertain results.
Critical Events	Apply thresholds, repeated detection, model agreement and quality rules before generating high/critical alerts.
Review & Override	Route uncertain events to reviewers; allow playback, correction, comments and controlled override while preserving original outputs.
Dashboard & Reports	Show live status, recent events, alerts, trends, waveform, spectrogram, analytics and downloadable reports.
Storage & Audit	Store users, audio metadata, predictions, alerts, reviews, model versions and audit records; detect duplicate audio.

2.2 Non-Functional Requirements

Quality Attribute	Target / Requirement
Performance	Classify uploaded audio up to 30 seconds within 8 seconds under normal conditions; live windows should produce a prediction within 3 seconds.
Scalability	Store/manage at least 20,000 sound-event records and support multiple concurrent users.
Usability	Responsive and understandable web interface for users, reviewers, operators and administrators.
Accuracy	Target >=85% overall test accuracy, macro F1 >=0.80, and >=85% recall for critical classes.
Availability	Target at least 99% uptime during required monitoring/evaluation hours.

Security & Privacy	Controlled access, secure audio storage, audit trail, visible microphone status, consent and configurable retention.

3. System Design

3.1 Application Architecture

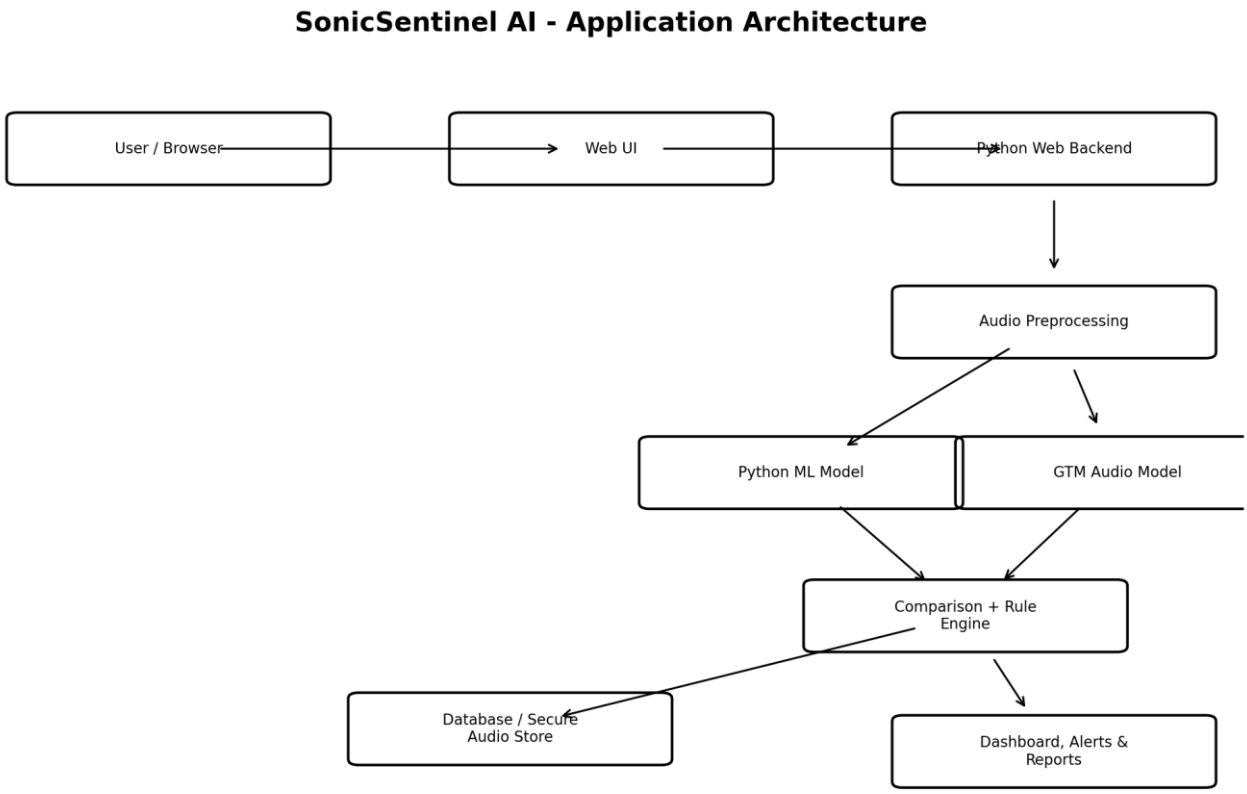


Figure 1. High-level application architecture.

3.2 Module Descriptions

Module	Responsibility
Authentication & Users	Login, profiles, roles and permissions.
Audio Capture / Upload	File upload, batch upload, microphone sessions and input status.
Audio Validation	Checks format, duration, channels, integrity, signal presence and permissions.
Preprocessing	Resampling, mono conversion, normalization, silence trimming, denoising, segmentation and padding/truncation.
Feature Extraction	Produces MFCC, Mel spectrogram, chroma, ZCR, RMS and spectral features for the Python pipeline.
Python Model	Trains/loads the selected classifier and generates class probabilities.
GTM Integration	Runs the separately trained GTM audio model on the same underlying audio segment.
Comparison & Rules	Compares predictions/confidence, checks quality, thresholds, repeated detections and critical-event rules.
Alerts & Review	Creates alerts, acknowledgement/escalation flow and manual-review queue.
Dashboard / Reporting	Displays live results, event history, analytics and exports.

Persistence / Audit	Stores metadata, model outputs, alerts, reviews, versions and user actions.

4. Database Design

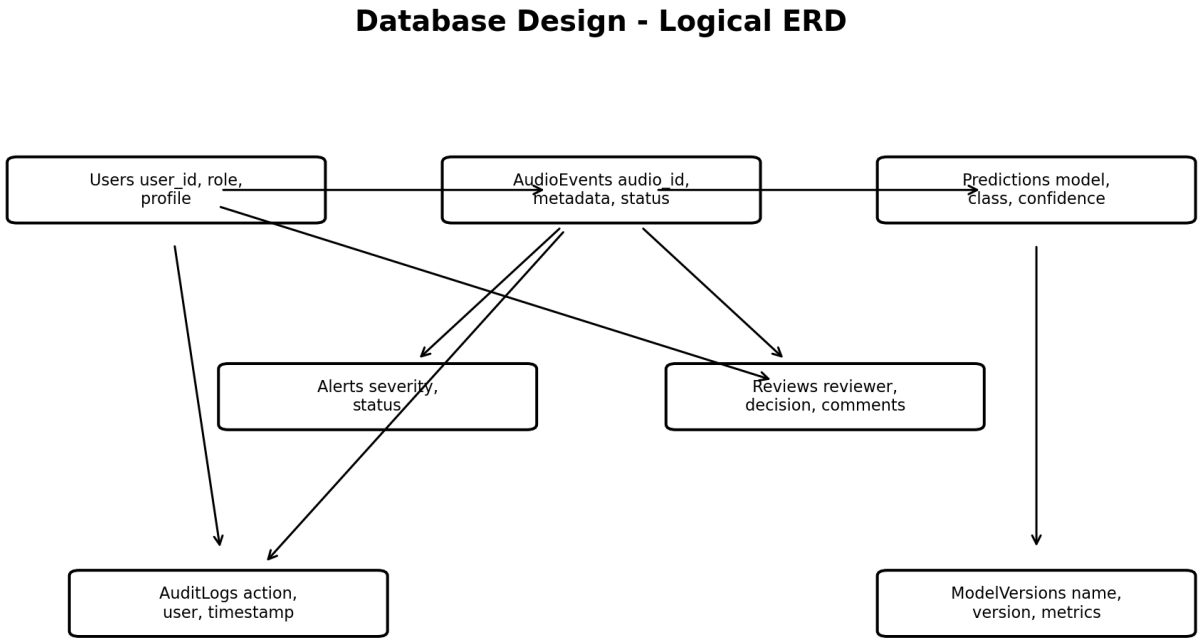


Figure 2. Logical database design / ERD.

4.1 Data Dictionary

Entity	Key Fields	Purpose
Users	user_id, name, email, password_hash, role, status, created_at	User identity and authorization.
AudioEvents	audio_id, filename, format, duration, sample_rate, channels, source, quality, status, created_at	Master record for uploaded or captured audio.
Predictions	prediction_id, audio_id, model_version_id, model_type, predicted_class, confidence_json, top2_margin	Stores Python/GTM outputs.
Alerts	alert_id, audio_id, severity, rule_triggered, status, acknowledged_by, created_at	Critical-event alert lifecycle.
Reviews	review_id, audio_id, reviewer_id, original_class, final_class, comments, decision_time	Manual review and controlled override.
ModelVersions	model_version_id, model_type, version, trained_at, dataset_version, metrics_json	Reproducibility and model tracking.
AuditLogs	audit_id, user_id, action, entity_type, entity_id, timestamp, details	Security and traceability.

4.2 Storage Notes

Audio files should be stored in controlled storage while the database stores metadata and references. Passwords should be stored only as secure hashes. Prediction confidence values may be stored as JSON or normalized child records. Retention rules should support deletion/archiving of old audio while preserving required audit information.

5. Diagrams

5.1 Data Flow Diagram

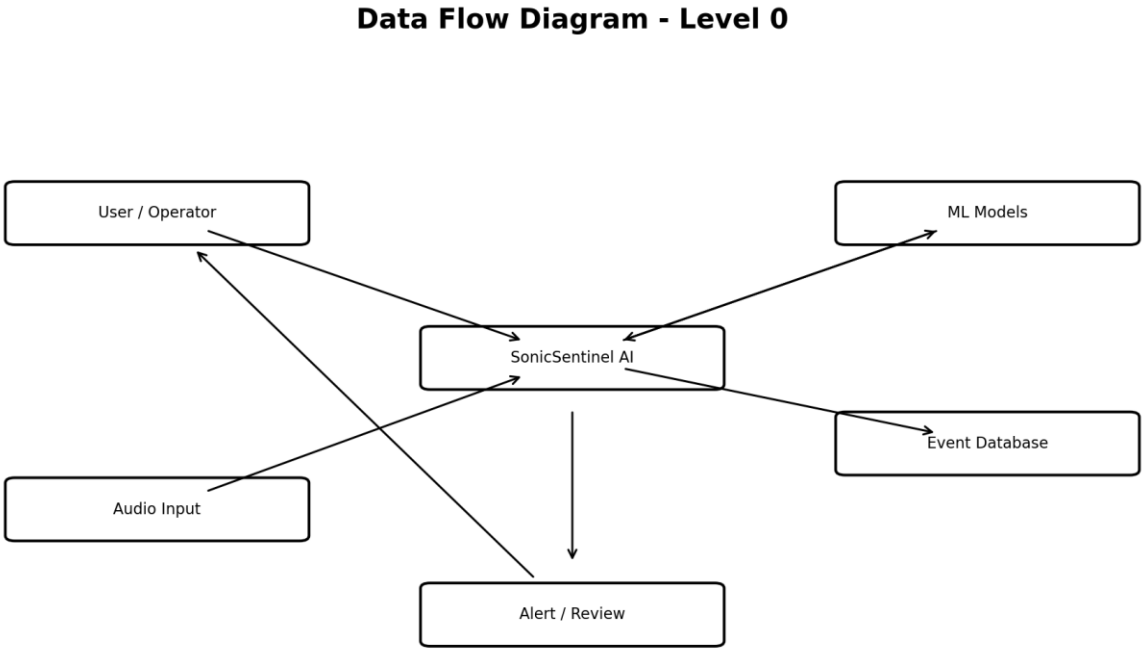


Figure 3. DFD Level 0.

5.2 Use Case Diagram

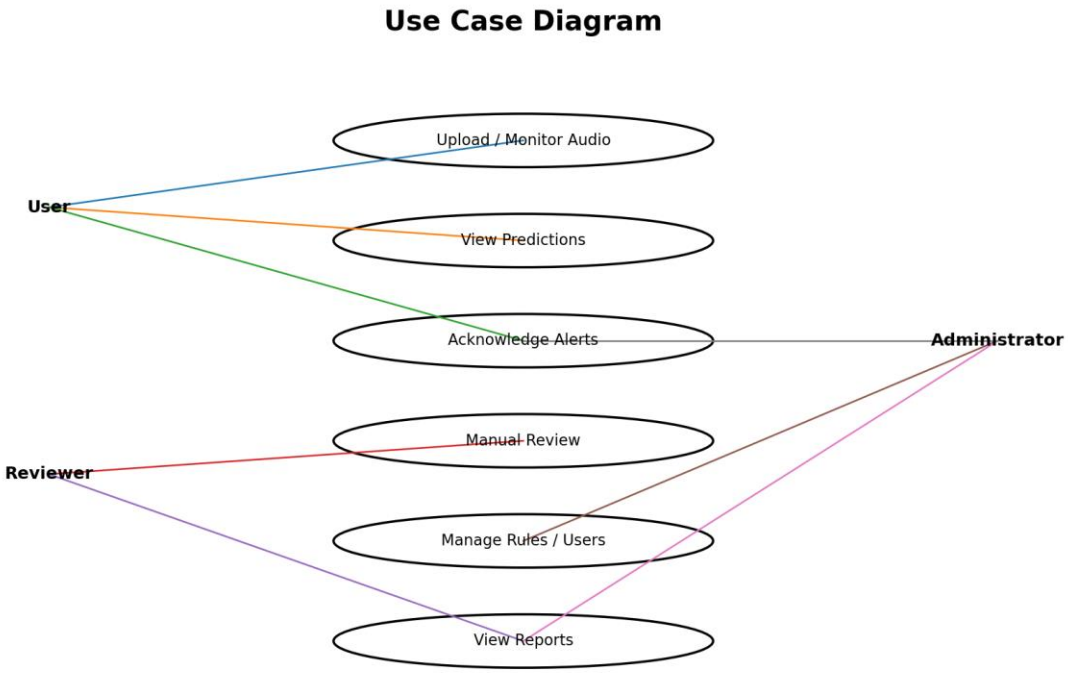


Figure 4. Main actors and use cases.

5.3 Activity Diagram

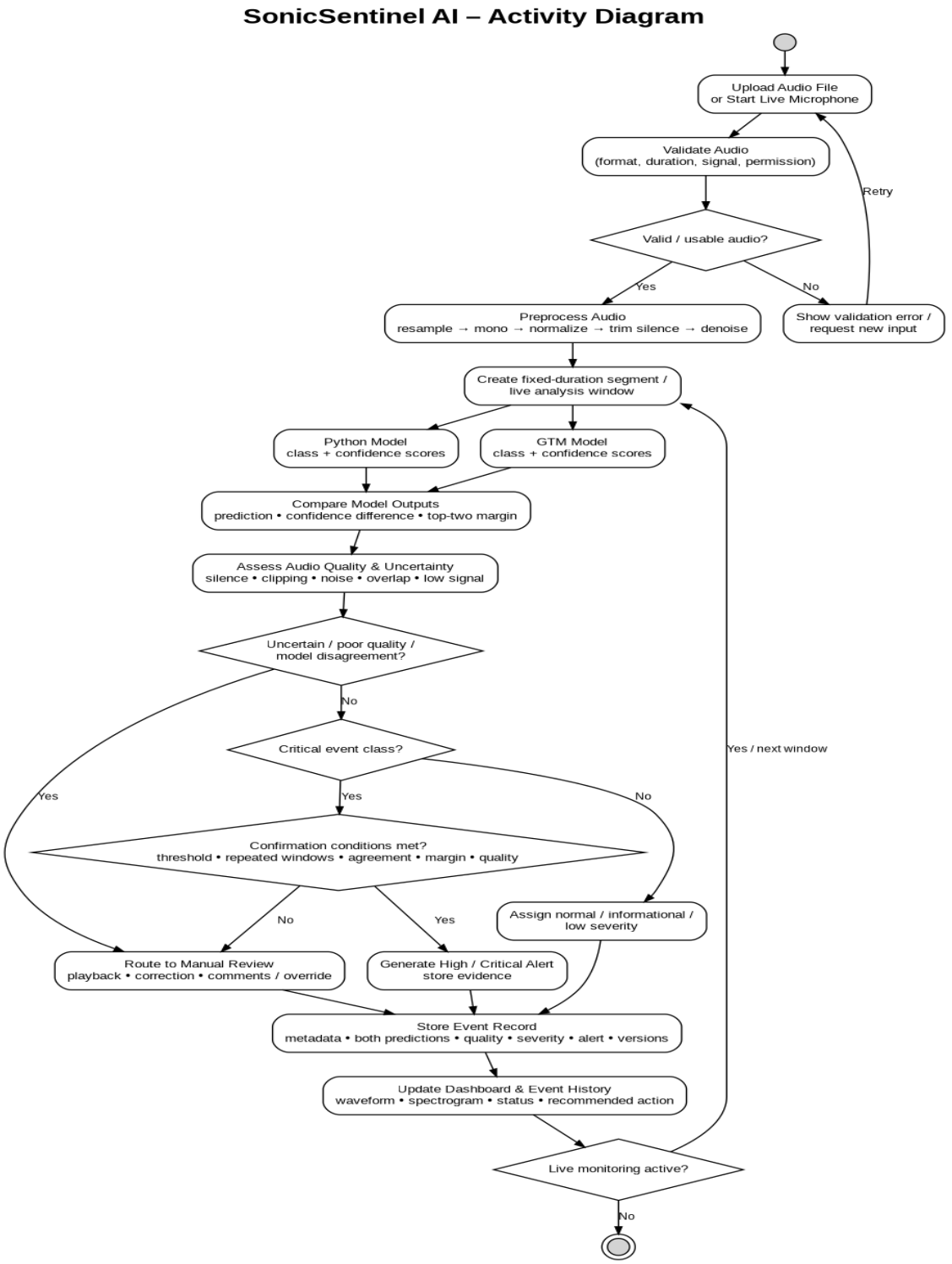


Figure 5. Main audio-analysis activity.

5.4 Sequence Diagram

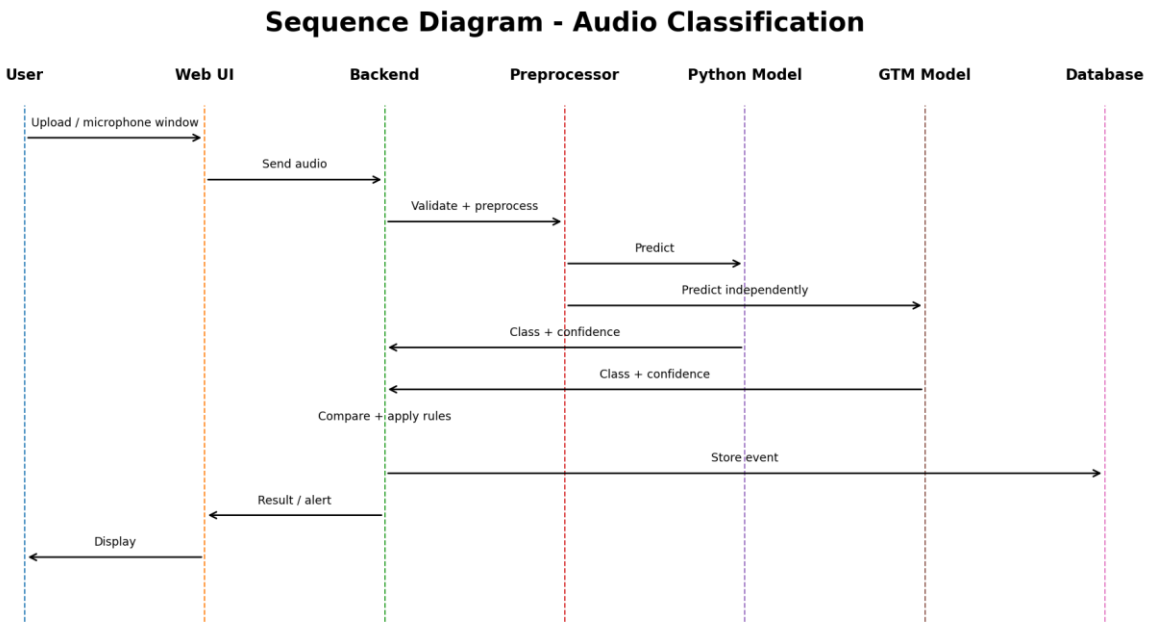


Figure 6. Audio classification sequence.

5.5 Decision Flow Diagram

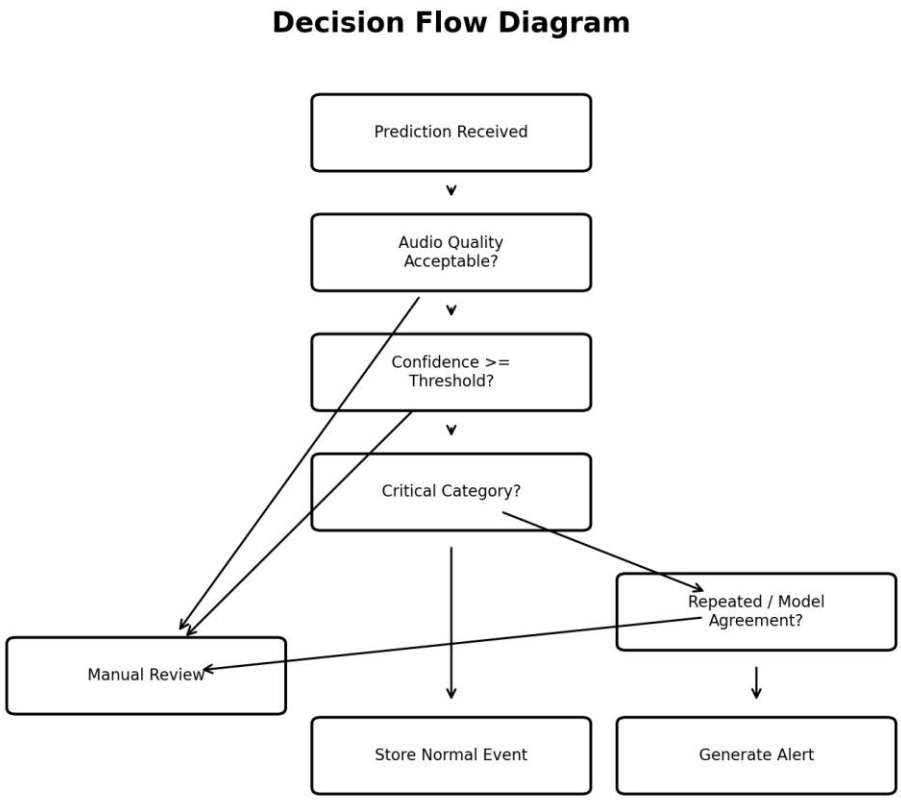


Figure 7. Simplified final-decision and alert flow.

6. Audio and AI Design

6.1 Audio-Processing Pipeline

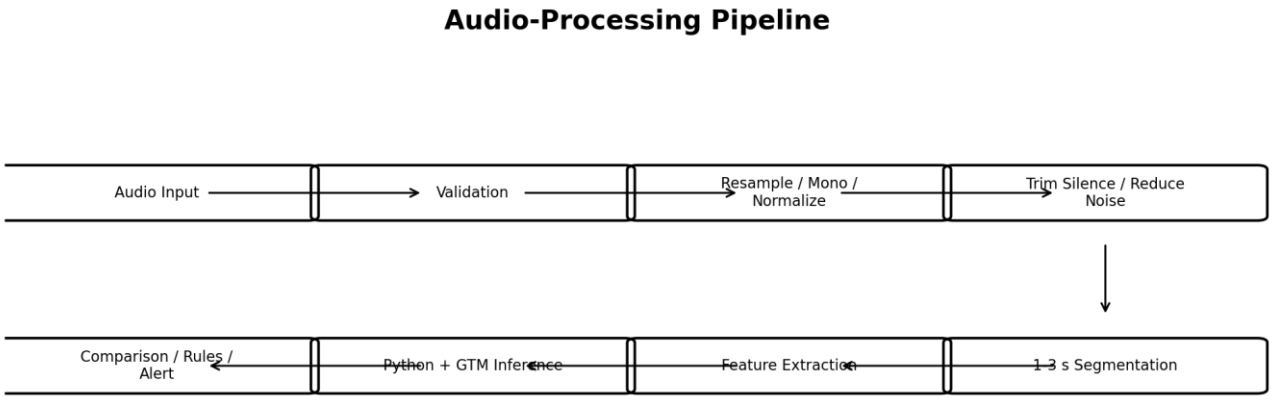


Figure 8. Audio-processing and inference pipeline.

6.2 Critical-Event Rule Design

A critical alert should not depend on one unstable prediction. The rule engine combines category, minimum confidence, top-two margin, audio quality, repeated detection, and optionally agreement between Python and GTM. Events such as Gunshot, Panic Scream, Person Asking for Help, Glass Breaking, Alarm/Siren, Aggression and serious Machinery Fault receive higher severity.

Condition	Typical Action
Low confidence / poor quality / model disagreement	Mark uncertain and route to manual review.
Critical class + sufficient confidence + confirmation	Generate high/critical alert and store evidence.
Critical class but no confirmation	Do not overstate certainty; retain event for review/repeated-window check.
Non-critical class	Store event and display normal/informational/low severity as configured.

6.3 Python Model Design

The Python pipeline uses the common labelled dataset, audio preprocessing, feature extraction and supervised classification. At least three suitable models should be trained and compared, for example SVM, Random Forest and a CNN/CRNN or another suitable model. The selected model must be based on test metrics, class-wise performance, critical-event recall and noise robustness rather than a hard-coded choice.

6.4 GTM Model Design

A separate Google Teachable Machine Audio Project uses the same ten class names and audio samples derived from the same underlying training recordings. GTM independently outputs a predicted class and confidence score for every class. Python predictions must never be passed into GTM.

7. Dataset and Training

7.1 Dataset Description

The project requires a minimum of 3,000 unique original clips, approximately 300 per mandatory class. Augmented clips do not count as unique originals. The dataset should vary by recording device, distance, environment, sound intensity, interference, echo, duration, clean/noisy condition and overlapping events.

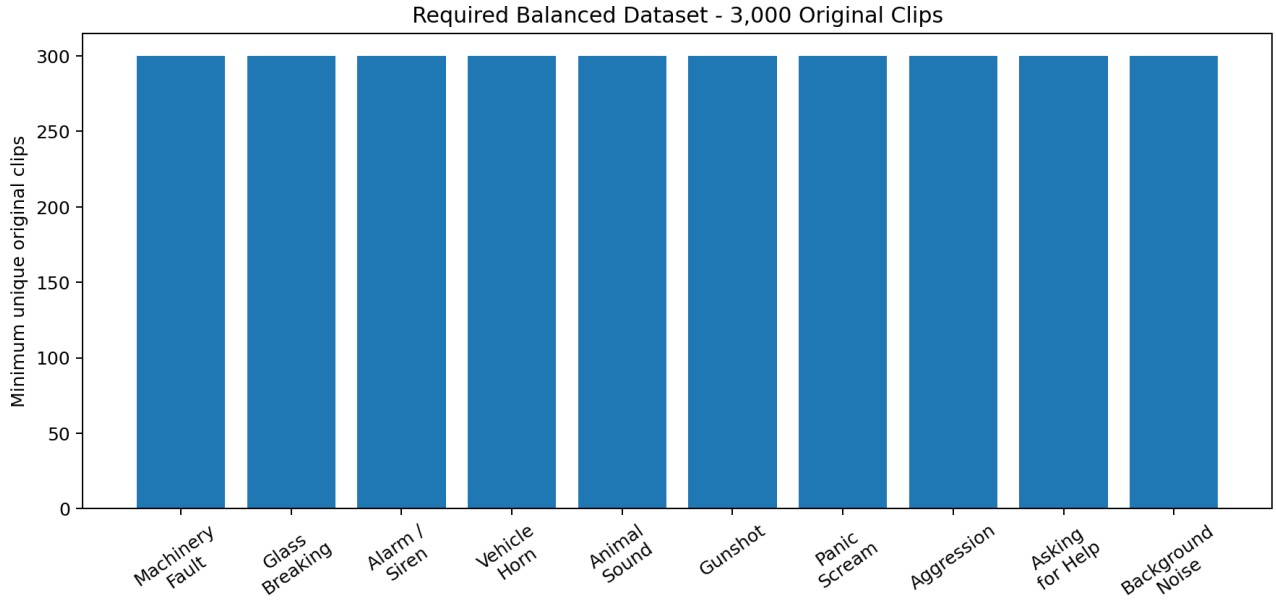


Figure 9. Minimum balanced class distribution required by the SRS.

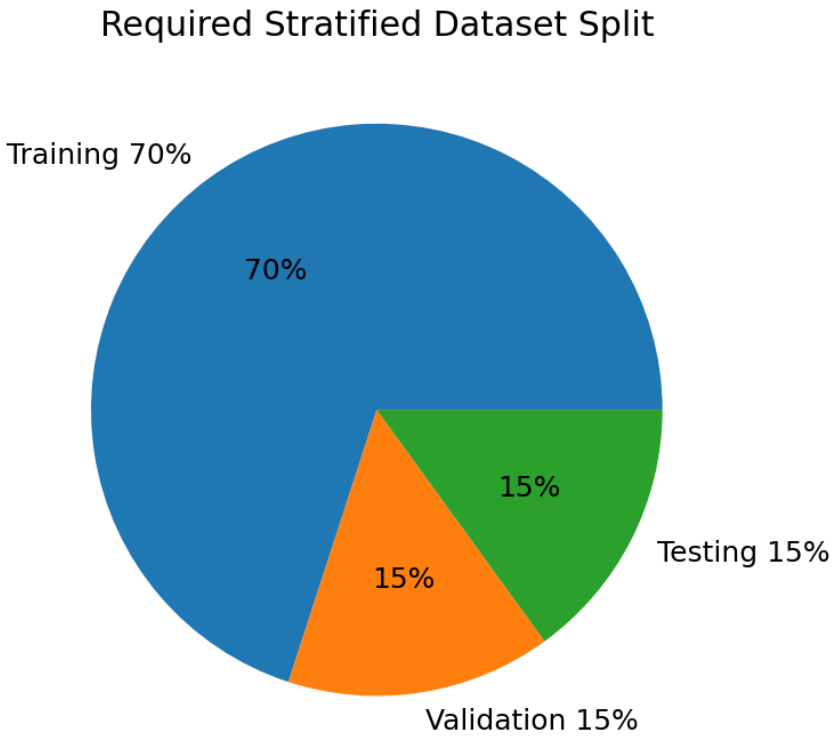


Figure 10. Required stratified 70/15/15 split.

7.2 Dataset-Collection Method

- Collect or ethically source labelled recordings for all ten classes.
- Assign each original recording a unique Audio ID.
- Record filename, class, duration, sampling rate, channels, environment, device, distance (if known), source, original/augmented status and split.
- Keep all segments derived from one original recording in the same split to prevent leakage.
- Use voluntary recordings, permitted synthetic generation, or ethically licensed sources.

7.3 Data Augmentation

Apply augmentation only to training data: background-noise addition, time shift, pitch shift, time stretch, volume adjustment, limited reverberation, distance simulation and recording-device simulation. Augmented samples remain linked to the original Audio ID and must stay in the original split.

7.4 Audio Preprocessing

Recommended sequence: decode -> validate -> resample -> mono conversion -> amplitude normalization -> silence trimming -> noise reduction -> fixed-duration segmentation -> padding/truncation -> feature/model-specific formatting.

7.5 Feature Extraction

The Python pipeline should extract MFCC, Mel spectrogram, chroma, zero-crossing rate, RMS energy, spectral centroid, spectral bandwidth, spectral roll-off, onset strength and tempo where relevant.

7.6 Python Model Training

- Train at least three suitable ML/DL models using only the training split.
- Tune hyperparameters systematically using validation data.
- Select the model using accuracy, precision, recall, F1, macro F1, confusion matrix, class-wise metrics, critical-event recall and noise robustness.
- Freeze the final model and record its version and dataset version before testing.

7.7 GTM Model Training

- Create a GTM Audio Project with exactly the same mandatory class names.
- Use training samples derived only from the common training recordings.
- Do not use validation or test recordings for training.
- Export/link the trained model and record its version for reproducible comparison.

8. Model Evaluation

This section should be completed with actual results after training. Values are intentionally left blank so the report does not invent model performance.

8.1 Evaluation Metrics

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Critical-class Recall
Python Model	TBD	TBD	TBD	TBD	TBD
GTM Model	TBD	TBD	TBD	TBD	TBD

8.2 Confusion Matrices

Insert one confusion matrix for the final Python model and one for GTM using the same unseen test recordings. Discuss the strongest diagonal classes and the most common confusions.

8.3 Model Prediction Comparison

Audio ID	True Class	Python Prediction	Python Conf.	GTM Prediction	GTM Conf.	Match?
Example / TBD	TBD	TBD	TBD	TBD	TBD	TBD

8.4 Confidence Comparison

For each test item calculate |Python top-class confidence - GTM top-class confidence| and also compare the top-two margin. Exact equality is not expected because the models are trained independently.

8.5 Noise-Robustness Analysis

Create controlled test groups such as clean audio, moderate background noise, low volume, echo/reverberation, distant source and overlapping sounds. Report accuracy/recall changes by condition and identify which classes degrade most.

8.6 False-Positive Analysis

Review cases where the system predicts a class that is not present. Record the true class, predicted class, confidence, audio quality, likely cause, whether both models agreed, and whether the rule engine correctly prevented or reduced an unnecessary alert.

8.7 False-Negative Analysis

Give special attention to missed critical events. Record the true critical class, incorrect prediction, confidence, noise/quality condition, model disagreement, and the improvement action such as more diverse data, threshold tuning or better preprocessing.

9. Testing, Security and Privacy

9.1 Testing Strategy

Test Type	Coverage
Unit Testing	Validation functions, preprocessing, feature extraction, thresholds, severity and helper functions.
Model Testing	Unseen test set, class-wise metrics, confusion matrices, critical recall and confidence calibration.
Integration Testing	Upload/microphone -> preprocessing -> both models -> rule engine -> database -> dashboard.
API Testing	Authentication, role checks, audio endpoints, reports, invalid payloads and error handling.
UI Testing	Responsive desktop/tablet/mobile layouts, playback, live status, filters and accessibility.
Robustness Testing	Noise, echo, low volume, partial events, different devices, overlap, re-encoding and distant sources.
Security Testing	Unauthorized access, failed logins, upload validation, permission checks and audit logging.
Performance Testing	30-second upload processing target, live prediction latency and concurrent usage.

9.2 Security

- Use secure authentication, password hashing, role-based authorization and least-privilege access.
- Validate file type/content and reject damaged or unsupported audio.
- Protect uploaded audio and generated reports with controlled access.
- Record logins, uploads, microphone sessions, predictions, alerts, reviews, overrides, exports and model updates.
- Track duplicate files using secure hashes and monitor failed logins/model failures/anomalies.
- Keep secrets and configuration outside source code where possible.

9.3 Privacy

- Clearly indicate when the microphone is active; never record secretly.
- Obtain permission/consent before live capture.
- Collect only audio and metadata needed for the project.
- Configure retention periods for audio and event records.
- Limit access to sensitive recordings and review/export functions.
- Use ethically sourced speech samples and avoid unnecessary personal information.

10. Limitations and Future Enhancements

10.1 Limitations

- Performance depends strongly on dataset diversity and label quality.
- Similar acoustic events can be confused, especially under noise or overlap.
- Browser microphones and recording devices produce different audio characteristics.
- GTM and Python confidence scores are not directly calibrated to be identical.
- The prototype is not a certified emergency-response system.
- Real-time performance depends on hosting resources, network conditions and client hardware.

10.2 Future Enhancements

- Add optional classes such as fireworks, vehicle backfire, explosion, drone, drilling/grinder, door impact and normal machinery.
- Introduce stronger transfer-learning audio models and confidence calibration.
- Add localization of events in long recordings and multi-label overlapping-sound detection.
- Support edge/offline inference for lower latency and privacy.
- Add notification channels, richer incident workflows and configurable escalation policies.
- Add multilingual help-phrase models and broader speech diversity after ethical data collection.
- Add model-drift monitoring, scheduled retraining and dataset version dashboards.

THANK YOU